Document Version: 1.0
Runner Version: 1.0.0
Test Version: 1.0.0

briteTest is a lightweight framework for defining, running, and reporting tests in C/C++ projects. It provides a compact macro-based Runner API, a function-driven Test API, fault-tolerant execution, and clear, structured reporting. The framework is designed for small to medium C projects that need reliable automated testing without the overhead of large toolchains or external dependencies.

Preface

BriteTest provides a Runner API and a Test API, each implemented with a single .h / .c pair with no external dependencies and requiring only a POSIX.1-2001 environment and a C99-compliant compiler.

For a list of other documents and the repository layout, see the Documentation Guide (`Documentation_Guide.md`).

For a glossary of terms, see the Glossary Reference (`Glossary_Reference.md`).

For a printer-friendly PDF file for this document, see `docs/pdf/briteTest.pdf`.

Document Version History

Document	Runner	Test	Date	Comment	Author/Editor
1.0	1.0.0	1.0.0	2026-06-11	Initial version.	Paul Sinclair

- The **Document** column records the document's version with the format `M.u` (Major, update).
- The **Runner** column records the Runner API version current at the time this document version was published and is defined by its `RA_VERSION` macro.
- The **Test** column records the Test API version current at the time this document version was published and is defined by its `RA_TEST_VERSION` macro.
- Both Runner and Test use the version format `"M.m.p"` (Major, minor, patch).
- `M` is the same for the Document, Runner, and Test versions.

The document's update version tracks released updates to this document and does not correspond to a minor or patch version. `u` increments when document changes are published in a release without a change to `M`, and it resets to `0` when `M` is incremented.

Table of Contents

1. Introduction

1.1. Key Strengths

- Lightweight design — minimal files, minimal API surface, easy to embed.
- Macro-based Runner API — simple orchestration with predictable control flow.
- Function-based Test API — tests are just C functions, easy to organize and debug.
- Fault-tolerant execution — protects the test suite from crashes and undefined behavior.
- Parallel and concurrent test execution,
- Clear reporting — readable summaries of passes, failures, and faults.
- Pure C implementation.
- Minimal footprint (single header + source cor the).
- Optional process-isolated execution.

1.2. Quick Start

A test executable consists of your tests as C/C++ expressions/functions, and the orchestrator and test group functions you write using the Runner API that manages the execution.

1. Copy 'runnerapi.h' and 'runnerapi.c' to your current directory:

```
cp /path/to/runnerapi.h .
cp /path/to/runnerapi.c .
```

2. Create a file named `test_quick.c` in the same directory.

3. Copy and paste the code into `test_quick.c`:

```c
#include "runnerapi.h"

// A simple test group function.

static RA_DECLARE_GROUP(test_quick)
{
  RA_INIT_GROUP(test_quick, 1);

  int a = 2;
  int b = 2;

  // 4 test assertions.
  RA_TEST(a == b, , 0);          // Pass
  RA_TEST(a + b == 4, ,  0);     // Pass
  RA_TEST(a - b == 1, , 0);      // Fail
  RA_TEST(RA_FAULT(1), , 0);     // Fault

  RA_RETURN;
}

// A simple orchestrator (main) function.
```

```
RA_DECLARE_ORCHESTRATOR(main)
{
  RA_INIT_ORCHESTRATOR(main, quick, 1);
  RA_PARSE_ARGS(2, "quick_test_report.txt");
  RA_OPEN_REPORT("Test Quick Report");

  // Single test category.
  RA_WRITE_RESULT(RA_GROUP(test_quick), "Quick tests");

  RA_CLOSE_REPORT("Note: This report is a very simple examplexx.\n"
                  "Note: Multiple test categories can be added using multiple\n"
                  "      test functions.\n"
                  "Note: Orchestrator (`main`) and test functions can be placed in\n"
                  "      individual modules (.c files).\n"
                  "Note: Parameters can be set to run tests in parallel, isolate\n"
                  "      a test to a separate thread or process, etc.\n"
                  "Note: The expression for RA_TEST can reference functions to\n"
                  "      provide a more complex test. A non-zero result indicates\n"
                  "      pass and a zero result indicates fail. If a fault occurs\n"
                  "      executing the expression, it is detected and counted in\n"
                  "      the report as a fault.\n"
                  "Note: Larger projects can place files in a more conventional\n"
                  "      layout (e.g., `include/` and `src/`, but this example keeps\n"
                  "      everything in your current directory for simplification.\n"
                  "Note: See root README.md for for additional API features.\n");
  RA_EXIT;
}
```

4. Build the executable `test_quick` in your current directory:

```
cc -std=c99 -Wall -Wextra -o test_quick test_quick.c runnerapi.c
```

5. Run it:

```
./test_quick
```

6. View `quick_test_report.txt` in your current directory:

```
less quick_test_report.txt   # Press 'q' to quit
```

Example of the report:

```
Test Report
                             Pass   Fail     Fault
-------------------------------------------------
1. Orchestrator                 4
2. Guard 1 and 2                6
-------------------------------------------------
                      Total    10
```


1.3. Requirements

Supported compilers, C standard level, and any platform notes.

1.4. Installation

How to add the Runner API to your project, including copying the header, adding the source file, or integrating via your build system.

2. Runner Framework and API

The Runner Framework (implemented using the Runner API) executes all test expressions (that implement the tests), aggregates results, and produces the report. This is the framework used by your test executable.

The Runner API provides typedefs, enums, macros, and functions for writing tests. Developers use this API to express expected behavior, group related tests, and define optional setup/teardown logic.

2.1. What the Runner Framework and API Does

The Runner framework coordinates test execution from the orchestrator down to individual test groups. It initializes the run, applies include and concurrency settings, executes the requested groups, captures pass/fail/fault results, and writes the final report.

The Runner API provides the macros and helper types used to define the orchestrator, declare test groups, control execution flow, and handle fault detection and reporting.

3. Running Tests

How to invoke the runner from `main()`, including optional parameters such as output paths or configuration flags.

4. Report Generation

How BriteTest collects pass/fail information and formats it for output.

5. Customization

Notes that you can build your own runner logic if you need custom ordering, filtering, or integration behavior.

6. Test Report

Describes the structure of the test report, failures, optional details, and how to consume it in tooling or CI.

7. Advanced Features and Topics

Filtering tests and custom report writers are available in the User Guide.

8. Test Expressions

How to write a test expression for `RA_TEST(expr)` and how expressions are evaluated.

9. Setup and Teardown

Optional per-group or per-test initialization and cleanup helpers.

10. Test Groups

How to organize related tests into groups for readability and logical structure.

11. Concurrent Tests

How to mark tests as concurrent and how Runner Framework handles parallel execution safely.

12. Isolation

Isolation controls how much separation is used when running tests.

- **Same-thread execution** is the default and has the lowest overhead.
- **Thread-isolated execution** keeps tests separated without switching to a separate process.
- **Process-isolated execution** provides the strongest containment for faults, crashes, aborts, and other disruptive failures.

Use the lightest mode that still gives you the safety you need for the code under test.

13. Test API

The Test API provides macros and functions for writing tests. Developers use this API to express expected behavior, group related tests, and define optional setup/teardown logic.

The Test API also includes execution/runtime, filesystem/path, environment, process-result, string/text, extended file operation, JSON data extraction, and resource management helpers.

See the Test Reference for a complete list of helpers provided by the API.

The following two sections provide examples of file and compare functions.

13.1 File Function Examples

These helpers cover file and directory-related functions in the Test API.

Examples include:

- Filesystem predicates: `ta_exists`
- File and directory operations: `ta_copy_file`
- Temporary and cleanup helpers: `ta_make_temp_dir`

13.2 Compare Function Examples

These helpers cover comparison and matching functions in the Test API.

Examples include:

- Path, file, and directory comparisons: `ta_compare_dirs`
- Metadata and binary comparisons: `ta_compare_path_metadata`
- JSON helpers: `ta_compare_json`
- Text and pattern helpers: `ta_compare_text_normalized`

SYNC_TEST_LINE